Hexabid Development Specification

Table of Contents

1. Executive Summary . . . 2
2. Technology Stack . . . 2
3. Module Organization . . . 2
   3.1. Core Modules (Pure Java, No Framework) . . . 2
   3.2. Input Adapters . . . 3
   3.3. Output Adapters . . . 3
   3.4. Contracts . . . 3
   3.5. Bootstrap . . . 3
4. Domain Models . . . 3
   4.1. Party (Authentication Domain) . . . 4
   4.2. Auction (Auctioning Domain) . . . 4
   4.3. Lot (Item for Sale) . . . 4
   4.4. Product (Catalog Domain) . . . 4
   4.5. Payment Domain . . . 5
5. Port Interfaces (Hexagonal Boundaries) . . . 5
   5.1. Input Ports (Use Cases) . . . 5
   5.2. Output Ports (Repositories and Services) . . . 6
   5.3. Auth Port . . . 6
6. Business Rules . . . 6
   6.1. Auction Creation . . . 6
   6.2. Bidding . . . 6
   6.3. Auction Closing . . . 7
7. API Specifications . . . 7
   7.1. REST API . . . 7
      7.1.1. Create Auction . . . 7
      7.1.2. Get Auction Details . . . 7
      7.1.3. Browse Auctions . . . 7
   7.2. WebSocket API . . . 7
      7.2.1. Connect . . . 7
      7.2.2. Place Bid . . . 8
      7.2.3. Subscribe . . . 8
   7.3. Authentication . . . 8
8. Event Specifications . . . 8
   8.1. Domain Events . . . 8
9. Payment Integration . . . 8
10. Scheduler . . . 9

11. Architecture Decision Records . . . 9
11.1. ADR-001: Hexagonal Architecture . . . 9
11.2. ADR-002: Contract-First . . . 9
11.3. ADR-003: Explicit Results . . . 9
11.4. ADR-004: Plugin Authentication . . . 9
11.5. ADR-005: Optimistic Locking . . . 9
12. Folder Structure . . . 9
13. Testing Strategy . . . 10
14. Build and Run . . . 10
14.1. Build . . . 10
14.2. Run with OAuth2 . . . 10
14.3. Run with Local Auth . . . 11
14.4. Integration Tests . . . 11

1. Executive Summary

This document provides comprehensive development specification for Hexabid, a reference auction platform built with hexagonal architecture. It covers the module structure, domain models, use cases, API contracts, and architectural decisions.

2. Technology Stack

- **Java 25** with latest features
- **Spring Boot 4.0.3** for composition and runtime
- **Spring MVC** for REST adapters
- **Spring WebSocket** with STOMP for real-time communication
- **Spring Security** for authentication
- **Spring Data JPA** for persistence
- **Spring Kafka** for event publishing
- **Maven** for build management

3. Module Organization

3.1. Core Modules (Pure Java, No Framework)

Module	Description
`quantity`	Unit and Quantity value objects for all measurements
`product`	Product catalog domain models
`inventory`	Inventory management domain

`auctions-core`	Core auctioning domain (Auction, Bid, Lot aggregates)
`auctions-auth-core`	Authentication domain (Party model)
`auctions-payment-core`	Payment processing domain

3.2. Input Adapters

Module	Description
`auctions-adapter-in-rest`	REST API endpoints (Spring MVC)
`auctions-adapter-in-ws`	WebSocket STOMP for bidding
`auctions-adapter-in-job`	Scheduler for closing expired auctions
`auctions-adapter-in-auth-oauth`	OAuth2/OpenID Connect (Google, GitHub)
`auctions-adapter-in-auth-local`	Local form login (development)

3.3. Output Adapters

Module	Description
`auctions-adapter-out-db`	JPA/Hibernate persistence
`auctions-adapter-out-kafka`	Domain event publishing
`auctions-adapter-out-kyc`	External KYC verification
`auctions-adapter-out-kyc-local`	Local KYC mock (development)
`auctions-payment-adapter-payu`	PayU gateway
`auctions-payment-adapter-p24`	Przelewy24 gateway
`auctions-payment-adapter-crypto`	Cryptocurrency payments
`auctions-payment-adapter-local`	Local payment mock (development)

3.4. Contracts

- `auctions-api-contract` - OpenAPI definitions for auction API
- `auctions-payment-api-contract` - OpenAPI definitions for payment API

3.5. Bootstrap

- `auctions-bootstrap` - Spring Boot application entry point

4. Domain Models

4.1. Party (Authentication Domain)

```
record Party(PartyId partyId, String displayName, Set<PartyRole> roles)
record PartyId(String value)
sealed interface PartyRole permits Seller, Buyer, Admin {}
```

The Party model represents a business participant independently of the authentication mechanism. Identity is derived from OAuth2/OpenID Connect provider or local credentials.

4.2. Auction (Auctioning Domain)

```
record Auction(
    AuctionId id,
    PartyId sellerId,
    LotId lotId,
    Price startingPrice,
    Price currentPrice,
    PartyId leaderId?,
    Instant endsAt,
    AuctionStatus status,
    List<Bid> biddingHistory,
    int version
)

enum AuctionStatus { OPEN, CLOSED }

record Bid(BidId id, PartyId bidderId, Price amount, Instant placedAt)
record Price(BigDecimal amount, String currency)
```

4.3. Lot (Item for Sale)

```
record Lot(LotId id, String title, String description, SellingMode
sellingMode)
enum SellingMode { WHOLE, DIVISIBLE, DIVISIBLE_ONLY }
```

4.4. Product (Catalog Domain)

```
record Product(ProductId id, String name, String description)
```

```
record ProductId(String value)
```

4.5. Payment Domain

```
record Account(AccountId id, String owner, List<AccountingEntry>
entries)
record AccountingTransaction(TransactionId id, AccountId from, AccountId
to, BigDecimal amount)
record AccountingEntry(EntryId id, AccountId accountId, BigDecimal
amount, EntryType type)
enum EntryType { DEBIT, CREDIT }
```

5. Port Interfaces (Hexagonal Boundaries)

5.1. Input Ports (Use Cases)

```
interface CreateAuctionUseCase {
    CreateAuctionResult execute(CreateAuctionCommand command, PartyId
seller)
}

interface PlaceBidUseCase {
    PlaceBidResult execute(PlaceBidCommand command, PartyId bidder)
}

interface BrowseAuctionsUseCase {
    AuctionBrowsePage execute(BrowseAuctionsQuery query)
}

interface FindAuctionDetailsUseCase {
    AuctionDetailsResult execute(AuctionId id)
}

interface CloseExpiredAuctionsUseCase {
    ClosedAuctionsResult execute(CloseExpiredAuctionsCommand command)
}
```

5.2. Output Ports (Repositories and Services)

```
interface AuctionRepository {
    Auction save(Auction auction)
    Optional<Auction> findById(AuctionId id)
    List<Auction> findExpired(Instant before)
}

interface AuctionEventPublisher {
    void publish(AuctionDomainEvent event)
}

interface KycClient {
    KycVerificationResult verify(PartyId partyId)
}
```

5.3. Auth Port

```
interface CurrentUserProvider {
    Optional<Party> current()
}
```

6. Business Rules

6.1. Auction Creation

1. Seller must be verified (KYC) to create auctions
2. End time must be in the future
3. Starting price must be positive with valid currency
4. Title and description are required

6.2. Bidding

1. Bid amount must exceed current price
2. Currency must match auction currency
3. User cannot bid on own auction
4. Auction must be OPEN and not expired
5. Optimistic locking prevents lost updates

6.3. Auction Closing

1. AUCTION_WON - Published when auction ends with winner
2. AUCTION_CLOSED_WITHOUT_WINNER - Published when no bids placed

7. API Specifications

7.1. REST API

7.1.1. Create Auction

```
POST /api/auctions
Content-Type: application/json

{
  "title": "Vintage Watch",
  "description": "Rare 1960s watch",
  "startingPrice": { "amount": "1000.00", "currency": "PLN" },
  "endsAt": "2026-04-15T18:00:00Z"
}
```

Response: 201 Created with AuctionView

7.1.2. Get Auction Details

```
GET /api/auctions/{auctionId}
```

Response: 200 OK with AuctionView, 404 Not Found

7.1.3. Browse Auctions

```
GET /api/auctions?status=OPEN&page=0&size=20&sort=endsAt,asc
```

Response: 200 OK with AuctionBrowsePage

7.2. WebSocket API

7.2.1. Connect

```
ws://localhost:8080/ws-auctions
```

7.2.2. Place Bid

```
SEND /app/auctions/{auctionId}/bids
{"amount": "1500.00", "currency": "PLN"}
```

7.2.3. Subscribe

```
SUBSCRIBE /topic/auctions/{auctionId}/bids
SUBSCRIBE /topic/auctions/{auctionId}/events
SUBSCRIBE /topic/auctions/{auctionId}/errors
```

7.3. Authentication

- OAuth2/OIDC with Google: `GET /oauth2/authorization/google`
- OAuth2/OIDC with GitHub: `GET /oauth2/authorization/github`
- Local login: `POST /login` (development profile)

8. Event Specifications

8.1. Domain Events

```
record AuctionWonEvent(AuctionId auctionId, PartyId winnerId, Price
winningPrice)
record AuctionClosedWithoutWinnerEvent(AuctionId auctionId)
record AuctionLeaderChangedEvent(AuctionId auctionId, PartyId
previousLeader?, PartyId newLeader, Price newPrice)
```

Events are published to: * Kafka topic: `auctions.events` * WebSocket: `/topic/auctions/{id}/events`

9. Payment Integration

Payment is processed when auction ends with a winner:

1. `AuctionWonEvent` triggers payment flow
2. `PaymentGatewayRegistry` selects gateway based on winner preference
3. Gateway adapters implement common interface

Supported gateways: * PayU (Polish payments) * Przelewy24 (Polish payments) * Cryptocurrency * Local mock (development)

10. Scheduler

`CloseExpiredAuctionsService` runs periodically: * Queries auctions where `endsAt < now()` and `status = OPEN` * Uses optimistic locking for concurrent updates * Resilient to conflicts (counts and skips)

11. Architecture Decision Records

11.1. ADR-001: Hexagonal Architecture

The system uses hexagonal architecture to: * Keep domain logic in pure Java * Allow framework-agnostic testing * Enable plugin-style adapters

11.2. ADR-002: Contract-First

API contracts are defined in OpenAPI before implementation: * Generated DTOs and delegates * Client libraries can be generated * Clear versioning

11.3. ADR-003: Explicit Results

Use cases return explicit results instead of throwing exceptions: * `CreateAuctionResult` with `CREATED` or `REJECTED(reason)` * `PlaceBidResult` with `ACCEPTED` or `REJECTED(reason)`

This prevents technical exceptions from leaking to adapters.

11.4. ADR-004: Plugin Authentication

Authentication adapter is selected at build time via Maven profile: * `auth-oauth` profile: Includes OAuth2 adapter * `auth-local` profile: Includes local login adapter

Production builds only include OAuth2.

11.5. ADR-005: Optimistic Locking

Auction aggregate uses optimistic locking (`version` field): * Concurrent bids are handled gracefully * `PlaceBidResult.CONCURRENT_MODIFICATION` returned instead of exception * No data loss or corruption

12. Folder Structure

```
src/main/java/com/acme/auctions/
├── core/                           # Shared archetypes
```

```
│   ├── party/model/                # Party, PartyId, PartyRole
│   ├── quantity/model/             # Unit, Quantity
│   └── common/                    # Preconditions, Result
├── auctions/
│   ├── core/                      # Core domain
│   │   ├── auctioning/
│   │   │   ├── model/             # Auction, Bid
│   │   │   ├── port/in/           # Use case interfaces
│   │   │   ├── port/out/          # Repository interfaces
│   │   │   ├── usecase/          # Service implementations
│   │   │   └── event/            # Domain events
│   │   └── lot/model/             # Lot, SellingMode
│   ├── auth-core/                  # Authentication domain
│   ├── api-contract/              # OpenAPI contract
│   ├── adapter-in-rest/           # REST adapter
│   ├── adapter-in-ws/             # WebSocket adapter
│   ├── adapter-out-db/             # JPA adapter
│   ├── adapter-out-kafka/         # Kafka adapter
│   └── bootstrap/                 # Spring Boot app
└── payment/
    ├── core/                      # Payment domain
    ├── adapter-payu/             # PayU adapter
    └── adapter-p24/              # Przelewy24 adapter
```

13. Testing Strategy

- **Unit Tests** - Domain logic in pure Java
- **Architecture Tests** - ArchUnit for dependency rules
- **Integration Tests** - Full application context

14. Build and Run

14.1. Build

```
mvn clean package
```

14.2. Run with OAuth2

```
mvn spring-boot:run -P auth-oauth
```

14.3. Run with Local Auth

```
mvn spring-boot:run -P auth-local
```

14.4. Integration Tests

```
mvn verify -DskipIntegrationTests=false
```